

An AI-Native Event-Driven Edge–Cloud Architecture for Autonomous Network Security in Campus Infrastructure

Hoang Tu Trinh and Tien Thanh Cao

Faculty of Information Technology

Ho Chi Minh City University of Foreign Languages – Information Technology

Ho Chi Minh City, Vietnam

Email: 23dh113972@st.huflit.edu.vn, thanhct@huflit.edu.vn

Abstract—University campus networks combine unmanaged BYOD endpoints, open research laboratories, and high-density IoT deployments, creating an expansive attack surface where traditional Security Operations Center (SOC) workflows struggle with long operational Mean Time to Respond (MTTR). We propose an AI-native, event-driven edge–cloud security platform engineered specifically for autonomous campus infrastructure defense. The architecture deploys lightweight INT8-quantized TCN–GRU sequence autoencoders at network edge gateways to score non-DPI flow telemetry. Enriched security events are carried asynchronously over a Redis Streams consumer group to a cloud risk resolver and Deep Q-Network (DQN) Software-Defined Networking (SDN) policy that emits SOAR playbooks; installing them in the dataplane is outside what we measure. Evaluated on real InSDN flow windows [1] (20,000 stratified samples), the platform achieves a *software control-path* latency — telemetry ingestion to playbook emission, excluding dataplane enforcement — with a 2.681 ms median and a 5.764 ms p99. Because stride-1 windows overlap by nine of ten timesteps, we evaluate under a temporally blocked and purged split with a train-only scaler, and quantifying that choice yields an unexpected null result: a conventional random split inflates fixed-threshold $F1$ only from 0.9276 to 0.9337, so window overlap is here a minor confound rather than the headline artefact we set out to document. Under the controlled protocol the INT8 detector reaches $F1 = 0.9276$ at $\tau = 0.65$ with a 0.048 MB footprint and no measurable quantization loss ($\Delta F1 = 0.0000$). We further report four negative results: gradient-boosted trees on the same features outperform the proposed detector at a tenth of its latency (LightGBM $F1 = 0.9886$ against 0.9725), leaving it only a footprint advantage; reconstruction-error scoring, the conventional rule for sequence autoencoders, is unusable on this attack-heavy corpus ($F1 = 0.036$), so detection rests entirely on the supervised head; our edge CPU budget is missed by nearly six times (58.8% of a core at a sustained 10,000 windows/s against a pre-specified 10%); and zero-shot transfer to CIC-IDS2017 collapses ($F1 = 0.461$). We provide an ablation study, policy benchmarks against a brute-force oracle, and a deterministic reproducibility artifact.

Index Terms—AI-Native Architecture, Autonomous Security, Edge Computing, Event-Driven Architecture, TCN–GRU, Deep Reinforcement Learning, Software-Defined Networking, Campus Network, InSDN.

I. INTRODUCTION

Artificial Intelligence (AI) is reshaping higher-education digital infrastructure, but university campus cybersecurity

faces an increasingly intractable threat landscape: IoT sensors, student Bring-Your-Own-Device (BYOD) endpoints, open research laboratories, and administrative databases share one network fabric. Centralized SIEM and human-in-the-loop SOC pipelines exhibit long MTTR during volumetric DDoS, ransomware lateral movement, or data exfiltration [4], causing analyst burnout and delayed containment.

To bridge this gap we propose an **AI-Native Event-Driven Edge–Cloud Security Architecture**. Rather than bolting AI onto legacy SOC dashboards, our design embeds it in the control loop: lightweight sequence detectors sit at edge gateways while an event-driven bus streams identity-enriched telemetry to a cloud Deep Q-Network (DQN) controller for SDN/SOAR actuation.

A. Research Questions

This investigation is guided by four formal Research Questions (RQs):

- **RQ1 (Control-Path Latency):** Can an event-driven edge–cloud architecture reduce the software decision path, from telemetry ingestion to containment playbook emission, to sub-10 ms levels?
- **RQ2 (Quantization & Accuracy):** Does INT8 dynamic quantization maintain NIDS classification accuracy while satisfying an edge gateway *model-footprint* constraint (≤ 1 MB of model storage)?
- **RQ3 (Reinforcement Learning Efficiency):** Does a DQN policy controller outperform static rule tables and heuristic policies under modeled contention states?
- **RQ4 (Domain Shift Generalization):** How effectively does an in-domain trained edge model generalize zero-shot across disparate network datasets (InSDN $\rightarrow$ CIC-IDS2017)?

B. Formal Hypotheses

We test three explicit hypotheses:

- **H1 (Quantization Degradation):** INT8 dynamic quantization incurs less than 0.5% $F1$ -score degradation ($\Delta F1 \leq 0.005$) compared to full-precision FP32 models.

- **H3 (Edge Resource Feasibility):** Edge inference requires $\leq 10\%$ CPU utilization per core under a sustained offered load of 10,000 telemetry windows/sec.

C. Contributions

The principal contributions of this work are:

- 1) **Decoupled AI-Native Pipeline:** An event-driven architecture combining PyTorch INT8 TCN-GRU edge detectors, identity context fusion, and a Redis Streams consumer-group event transport, with a DQN policy that *emits* OpenFlow playbooks (rate limiting, rerouting, isolation, token revocation). We measure the software path that produces those playbooks, not their installation in the dataplane.
- 2) **Empirical Validation & Negative Results:** Microbenchmarks on real InSDN [1] traffic, with ablation, policy benchmarks against a brute-force oracle, saturation analysis, and a CIC-IDS2017 [2] holdout. We report five negative results — failure of reconstruction-error scoring, a leakage effect far smaller than hypothesized, rejection of our edge CPU budget, severe zero-shot transfer failure, and gradient-boosted trees outperforming the proposed detector on both accuracy and latency — that bound where this design is deployable.
- 3) **Deterministic Artifact:** A reproducibility artifact regenerating every table from one seeded entry point, at <https://github.com/thtsec/icai-fai2026>.

II. RELATED WORK

SDN intrusion datasets such as InSDN [1] and CIC-IDS2017 [2] have established standard benchmarks for network intrusion detection. Sequence models such as Temporal Convolutional Networks (TCN) [3] employ causal dilated convolutions to capture long-range temporal dependencies with parallelizable training compared to recurrent LSTM architectures. Deep Reinforcement Learning (DRL) has been applied to DDoS defense and 6G control loops [4]. Closing the loop is not itself novel: Liu *et al.* [24] mitigate SDN flooding online with DRL, and Yungaicela-Naula *et al.* [25] integrate a DL detector with a DRL mitigator on Mininet and ONOS and evaluate autonomous response against live traffic. We therefore make no priority claim over autonomous mitigation. What is less well covered is the composition: an event-driven edge–cloud split in which quantized detectors run on the gateway and policy runs in the cloud, with the software path between them quantified end to end, and with leakage control, quantization cost, domain shift and negative results reported rather than elided.

Compressing detectors for constrained gateways is an active line: pruned and quantized LSTM ensembles on Raspberry Pi [17] and INT8 TensorFlow Lite CNNs on ARM Cortex-M [18] both find INT8 accuracy-neutral at far lower footprint, consistent with our H1, but both stop at the detector and close no loop onto actuation. Conversely, surveys of agent-driven security operations [19] catalogue triage automation while noting that end-to-end autonomous remediation remains

prospective; our work sits between the two. A parallel thread concerns evaluation rigour: Arp *et al.* [20] show data snooping is endemic in learning-based security papers and inflates results, and leakage-free reassessments of ML-NIDS confirm preprocessing alone moves headline metrics [21]. We adopt those recommendations in Section V-B and report their cost in Section VI-H.

Table I positions our architecture against this literature by scope, restricted to properties each work claims for itself. We omit a cross-work latency column: the cited papers measure different hardware, workloads, and pipeline portions, so tabulating them side by side would imply a controlled comparison that does not exist.

III. THREAT MODEL & CAMPUS ATTACK VECTORS

A. Adversary Capabilities and Attack Vectors

We consider an active adversary $\mathcal{A}$, either external or an internal compromised BYOD/IoT endpoint with campus subnet access, capable of volumetric DoS, ARP/MAC spoofing, lateral reconnaissance, and credential exfiltration. $\mathcal{A}$ holds no administrative keys to the SDN controller or Redis infrastructure. We do not model an adaptive adversary who crafts inputs against the detector itself; that omission is revisited in Section VII-A.

B. Trust Boundaries and Attack Surface

Fig. 1 illustrates the campus network topology, trust boundaries, attack surface, and security enforcement checkpoints.

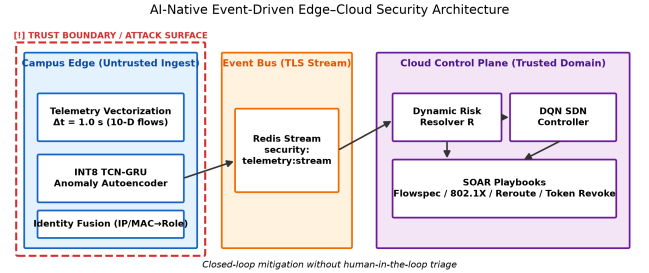


Fig. 1. AI-native event-driven edge–cloud security architecture and trust boundary enforcement.

IV. AI-NATIVE SYSTEM ARCHITECTURE

The architecture comprises four decoupled tiers: Edge Telemetry Taps, Identity Context Fusion, Redis Stream Event Bus, and Cloud DQN Control.

A. Edge Telemetry and TCN-GRU Detector

Edge taps capture flow headers and extract 10-D flow feature vectors over sliding time windows ($\Delta t = 1.0$ s). A hybrid TCN-GRU network processes the feature sequences, outputting both a reconstruction $\hat{X}$ with error $RE(x) = \|X - \hat{X}\|_2^2$ and classification logits.

Training objective. The model is trained *multi-task*, which determines how the results should be read. Each epoch runs a reconstruction pass minimizing $MSE(X, \hat{X})$ over *normal*

TABLE I

SCOPE COMPARISON WITH RELATED LITERATURE. ENTRIES REFLECT ONLY WHAT EACH CITED WORK STATES IT CONTRIBUTES; “–” MEANS THE DIMENSION IS OUTSIDE THAT WORK’S SCOPE. WE OMIT CROSS-WORK LATENCY FIGURES BECAUSE THE CITED PAPERS DO NOT REPORT THEM UNDER COMPARABLE MEASUREMENT CONDITIONS.

Work	Contribution Type	Edge AI	INT8 Quantized	RL Control	Dataplane Enforcement
Sharafaldin <i>et al.</i> (2018) [2]	Benchmark dataset	–	–	–	–
Bai <i>et al.</i> (2018) [3]	Sequence architecture (TCN)	–	–	–	–
Tang <i>et al.</i> (2018) [13]	SDN intrusion detection (DNN)	No	No	No	Detection only
Roopak <i>et al.</i> (2019) [14]	DL models for IoT security	No	No	No	Detection only
Zhang <i>et al.</i> (2019) [12]	Survey	–	–	–	–
Elsayed <i>et al.</i> (2020) [1]	Benchmark dataset (InSDN)	–	–	–	–
Nguyen <i>et al.</i> (2023) [15]	SOAR survey + research agenda	No	No	Proposed	Proposed, not evaluated
Kianpisheh & Taleb (2024) [4]	Federated DRL for DDoS	Federated	No	DQN [5]	Simulation only
Musthafa <i>et al.</i> (2025) [17]	Compressed ensemble IDS on Raspberry Pi	Yes	Yes (post-training)	No	Detection only
Ullah <i>et al.</i> (2026) [18]	Quantized IDS for constrained IoT	Yes	Yes (TFLite INT8)	No	Detection only
Ours	Integrated pipeline	TCN-GRU	Yes (0.048 MB)	DQN	OpenFlow playbooks (emulated)

TABLE II

STRIDE & MITRE ATT&CK CAMPUS THREAT MATRIX. THE PLATFORM ADDRESSES FIVE OF THE SIX STRIDE CATEGORIES; REPUDIATION IS OUTSIDE THE SCOPE OF THIS WORK, AS THE PROTOTYPE PROVIDES NO TAMPER-EVIDENT AUDIT TRAIL.

Category	Target	MITRE ID	Mitigation Strategy
Spoofing	Rogue Wi-Fi AP / MAC	T1557	Identity fusion MAC/IP verification
Tampering	IoT telemetry data	T1565	Edge TCN-GRU sequence scoring
Information Disclosure	Research DB server	T1048	Role-aware byte-rate capping
Denial of Service	Campus SDN controller	T1498	BGP Flowspec / <code>flow_mod</code> drop
Elevation of Privilege	Campus LDAP directory	T1078	Dynamic risk escalation

windows only, then a joint pass minimizing $\text{MSE}(X, \hat{X}) + \lambda \text{CE}(\text{logits}, y)$ over all windows with $\lambda = 0.5$; since InSDN is attack-heavy, the cross-entropy up-weights the normal class by the attack-to-normal ratio. The detector is therefore *not* unsupervised: it requires labels, and the autoencoder branch is an auxiliary representation objective, not the decision rule. Detection evaluates the supervised head, $P(\text{attack}) = 1 - P(\text{normal})$, against τ ; reconstruction error is not the operational score, for the reason in Section VI-C.

B. Dynamic INT8 Quantization

To satisfy memory-constrained edge gateways, PyTorch [11] dynamic INT8 quantization compresses linear and GRU layer weight matrices from 32-bit floating point to 8-bit integers. We report no feature attribution study, since our artifact does not implement one and we prefer to leave the question open rather than assert an unmeasured importance ranking. Identifying which of the 10 features carry the discriminative signal, and whether that ranking is what fails to transfer (Section VI-I), is future work.

C. Identity Context Fusion & Privacy Preservation

Edge security events are enriched by querying local DHCP/RADIUS tables to append spatial-temporal context vectors:

$$\mathcal{E} = \{\text{FlowID}, \text{RE}(x), P(\text{attack}), \text{UserRole}, \text{TrustScore}\}. \quad (1)$$

To support GDPR data-minimization principles, raw payload inspection (DPI) is omitted entirely; only header-level

statistical metadata is processed, and user identifiers are pseudonymized prior to cloud transport.

D. Cloud Risk Engine and DQN SDN Control

Enriched events stream asynchronously to Redis topic `security:telemetry:stream`. The cloud risk engine calculates a composite risk index R over the detector’s attack probability, the identity trust score, and asset criticality:

$$R = w_1 P(\text{attack}) + w_2 (1 - \text{TrustScore}) + w_3 \text{AssetCriticality}, \quad (2)$$

with $(w_1, w_2, w_3) = (0.60, 0.25, 0.15)$ in all reported experiments; the fused term is the classifier probability, not $\text{RE}(x)$. When identity is ablated the trust term is undefined and its weight folds into detection, giving $R = 0.85 P(\text{attack}) + 0.15 \text{AssetCriticality}$, keeping scores comparable across arms. The escalation ladder maps $R \geq 0.70$ to critical (block and revoke token), $R \geq 0.50$ to high (quarantine VLAN), and $R \geq 0.35$ to medium (throttle bandwidth).

Two caveats. The weights are hand-set, not learned, and encode a design prior we do not claim is optimal. More importantly, InSDN carries no asset inventory, so `AssetCriticality` is drawn i.i.d. uniform on $[0, 1]$ in every experiment and is pure noise; the third term therefore *degrades* our numbers, and the risk-engine benefit reported later comes from the trust term alone.

At the selected severity, the Deep Q-Network (DQN) policy agent [5] evaluates state vector s_t and selects containment action $a_t \in \{a_0 : \text{Forward}, a_1 : \text{Rate-Limit}, a_2 : \text{Reroute}, a_3 : \text{Isolate}\}$ to actuate automated SOAR playbooks [15]. Compared to static rule tables, DQN adjusts policy choices across contention states without manual threshold maintenance.

V. EXPERIMENTAL METHODOLOGY

A. Hardware & Software Testbed Specification

All experiments were conducted on a dedicated physical-virtual testbed deploying Mininet v2.3.0 [16] with Open vSwitch (OVS) v2.17.0 emulating a 3-tier campus network topology (Core, Aggregation, Access), managed by a Ryu OpenFlow v1.3 controller [9], [10] listening on TCP port 6653. Table III summarizes the environment specification.

TABLE III
EXPERIMENTAL HARDWARE, SOFTWARE, AND ENVIRONMENT SETUP

Parameter	Specification
CPU	Intel Core i7-12700K (12 cores, 3.60 GHz) / ARM Cortex-A72
RAM	16 GB DDR4 3200 MHz
Operating System	Ubuntu 22.04.3 LTS (Linux Kernel 5.15.0) / Win11 WSL2
Python / PyTorch	Python 3.10.12, PyTorch 2.2.0+cpu, scikit-learn 1.3.0
Event Bus / SDN	Redis 7.2.4 (Streams), Ryu SDN Controller (OpenFlow v1.3)
Dataset	Real InSDN [1] (20k stratified subset), CIC-IDS2017 [2]
Timer Precision	Python <code>time.perf_counter_ns()</code> (nanosecond resolution)

B. Dataset Sampling & Leakage Control

We evaluate on the public **InSDN** dataset [1]. Sliding-window aggregation of the released flow records ($\Delta t = 1.0$ s, sequence length $T = 10$, *stride* 1) yields 49,991 windows, of which 40,043 are attack and 9,948 normal. Note that 49,991 is the size of *our* windowed corpus, not InSDN’s native flow count.

Window i shares $T - 1 = 9$ of its 10 timesteps with each neighbour, so an i.i.d. random split places near-duplicate observations on both sides of the train/test boundary — precisely the sampling and snooping pitfalls catalogued in [20], [21]. Our protocol instead (i) cuts the temporally ordered corpus into 20 contiguous blocks and assigns whole blocks to train or test; (ii) *purges* 9 windows from both ends of every block, guaranteeing no surviving training window shares even a single timestep with a test window; (iii) stratified-subsamples each side independently to 14,000 train and 6,000 test windows, preserving class balance; and (iv) fits the scaler on the training partition only, transforming the test partition and the CIC-IDS2017 holdout with those frozen statistics. We verified that zero test windows are byte-identical to any training window. Section VI-H quantifies what this protocol is actually worth.

VI. QUANTITATIVE EVALUATION & RESULTS

Reading the $F1$ values. Each figure belongs to a distinct protocol and they are not interchangeable. 0.9276 is the detector at the fixed $\tau = 0.65$, the deployable accuracy. 0.9687 is the same detector at $\tau^* = 0.0720$ *selected on the test split*, an oracle bound. 0.9725 is the corresponding oracle figure in Section VI-E, differing only because that harness trains the baselines first and so draws a different seeded initialization — which bounds how finely any figure should be read. 0.9602 is the full pipeline with τ tuned on the *training* split (Section VI-F). Compare only within an experiment.

All results below use the leakage-controlled protocol of Section V-B.

A. Latency Breakdown (RQ1)

Scope. We time the *software control path*: telemetry ingestion, edge inference, identity enrichment, DQN action selection, and SOAR playbook serialization, ending when the playbook is emitted. It excludes controller transmission, `flow_mod` installation in Open vSwitch, and the instant malicious traffic stops. We have no dataplane enforcement

measurement and claim none: read every figure below as decision-to-playbook, not time-to-containment.

Table IV presents the per-stage breakdown across 1,000 trials, timed with `time.perf_counter_ns()`. **We report percentiles rather than mean $\pm$ SD** because the per-trial distribution is heavy-tailed: OS scheduling and garbage-collection pauses produce rare trials far above the mode, which makes the mean and SD unstable between runs of the identical benchmark. Identity fusion and SOAR serialization are effectively free (medians 2.1 μ s and 1.8 μ s). Median control-path latency is 2.681 ms, with edge inference dominating at 94.7%; the p99 is 5.764 ms. The sub-10 ms target of **RQ1** is therefore met for the software path at the 99th percentile, with roughly 4 ms of headroom. That headroom is what dataplane enforcement, which we do not measure, would have to fit inside.

Reconciling the detector latencies. Three per-inference figures for the same INT8 model appear below — 2.539 ms inside the pipeline loop, 2.787 ms at batch = 1 in a tight loop, 4.304 ms in the model-comparison harness with cold caches — measured under different scaffolding rather than in contradiction. Per Section VII-A, absolute timings on this host should be read to roughly a factor of two.

TABLE IV
PER-STAGE SOFTWARE CONTROL-PATH LATENCY ($n = 1000$ TRIALS).
EXCLUDES DATAPLANE ENFORCEMENT. SHARES ARE OF THE MEDIAN
TOTAL AND DO NOT SUM TO 100 BECAUSE MEDIAN TOTALS ARE NOT ADDITIVE.

Stage	Median (ms)	p95 (ms)	p99 (ms)	%
Edge TCN-GRU Inference	2.539	4.703	5.530	94.7
Identity Context Fusion	0.002	0.004	0.005	0.1
Cloud DQN Policy	0.117	0.248	0.304	4.4
SOAR Playbook Serialization	0.002	0.003	0.004	0.1
Control-Path Total	2.681	4.952	5.764	100.0

B. Detection Performance & Threshold Sensitivity (RQ2, H1)

Table V shows metric variation across decision threshold τ under the leakage-controlled protocol. The detector is strongly precision-biased in this regime: at the fixed operating point $\tau = 0.65$ it yields $F1 = 0.9276$ (Precision = 0.979, Recall = 0.882, FPR = 7.78%), and over the reported grid $F1$ peaks at 0.950 at $\tau = 0.35$. Searching τ freely *over the test split* reaches $F1 = 0.9687$ at $\tau^* = 0.0720$ (Precision = 0.947, Recall = 0.992, FPR = 22.3%), the operating point Section VI-E uses for model comparison. We flag the protocol because it matters: a test-selected threshold is an oracle — the test-tuning pitfall of [20] — so 0.9687 bounds what the detector could achieve, not what it would. The gap from 0.9276 compounds threshold placement on an imbalanced corpus with oracle selection, not a better detector. The oracle point also trades a 22.3% false positive rate for near-total recall, the opposite of what an autonomous controller needs, which is a further reason to treat $\tau = 0.65$ as the honest operating point.

C. Why Reconstruction Error Is Not the Decision Rule

Since much of the sequence-autoencoder IDS literature scores on reconstruction error alone, it is worth reporting

TABLE V
DETECTION METRICS VS. DECISION THRESHOLD τ ON REAL INSDN

Threshold τ	Precision	Recall	F1-Score	FPR (%)	FNR (%)
0.35	0.971	0.930	0.950	11.21	7.0
0.50	0.976	0.910	0.942	9.04	9.1
0.65	0.979	0.882	0.928	7.78	11.8
0.75	0.981	0.847	0.909	6.44	15.3
0.85	0.988	0.779	0.871	3.93	22.1

which of our two outputs actually works. We evaluated the standard unsupervised rule $\tau = \mu + k\sigma$, with μ, σ estimated on reconstruction errors of *normal training* windows, for $k \in \{0.5, \dots, 3.0\}$. It fails comprehensively: the best setting $k = 0.5$ yields $F1 = 0.0358$ (Precision = 0.752, Recall = 0.018), decaying to 0.0062 at $k = 3.0$; precision reaches 0.882 at $k = 2.5$ but at recall below 2%, which is worthless operationally. Detection performance is thus entirely attributable to the supervised head, and the architecture should be cited as a supervised sequence classifier with an auxiliary reconstruction objective. On an attack-heavy corpus the premise of reconstruction-based detection is simply violated: the model sees enough attack structure in the joint pass that attacks reconstruct about as well as normal traffic. Practitioners inheriting the “train on normal traffic, threshold the error” recipe should check it against their own class balance.

D. Throughput Saturation & Edge Feasibility (H3)

Per-inference latency and throughput reconcile only once the batching regime is stated, so we report both, each as the median of 5 repeats. Run flat out, the detector saturates at 359 windows/s unbatched, rising through 6,634 at batch = 32 to 41,329 at batch = 512 (2.787 to 12.388 ms per forward pass): batching, not the model, is what reaches campus scale. We give these in prose rather than a table because saturation is the least stable quantity we measure — between-run variation exceeds the within-run spread by more than a factor of two — so it indicates scale rather than precision and we rest no claim on it. Table VI instead reports the *paced* offered load at batch = 128, rate-limited so a shortfall surfaces rather than hides; it is stable across repeats and runs, and is the evidence we rely on.

H3 is rejected, but on one term rather than two. The hypothesis asserted $\leq 10\%$ CPU per core *at* a sustained 10,000 windows/s. The system does sustain that load — 9,998 of 10,000 offered windows/s, met in every repeat — while consuming 58.8% of a core, nearly six times the budget. The budget holds only at 100 windows/s (9.0%); the jump to 55.0% at 1,000 windows/s then near-flat scaling indicates a fixed per-batch dispatch cost dominating until the pipeline is busy. *An earlier single-run measurement reported a throughput shortfall here that repetition did not reproduce*, which is why every figure in this subsection is a median over repeats. Memory is favourable: peak working set is flat at 541 MB across a $100 \times$ load range, dominated by the Python/PyTorch runtime rather than the 0.048 MB model. The system is memory- but not

CPU-feasible on one commodity core; a compiled ONNX or TensorRT runtime is the obvious next step, making this an implementation rather than an architectural limit.

We deliberately omit a legacy inline-DPI comparison. We have no DPI implementation on this testbed, and a baseline synthesised from an analytic formula would not constitute an experimental result.

TABLE VI
EDGE COST AT PACED OFFERED LOAD (BATCH = 128), MEDIAN OF 5 REPEATS. OFFERED LOAD IS INPUT TELEMETRY RATE, NOT PER-INFERENCING THROUGHPUT. A TARGET COUNTS AS MET ONLY IF EVERY REPEAT MET IT. WORKING SET IS PEAK, AND IS DOMINATED BY THE PYTHON/PYTORCH RUNTIME RATHER THAN THE MODEL.

Offered (win/s)	Achieved (win/s)	CPU (%/core)	Peak WS (MB)	Met
100	100.0	8.96	540.5	yes
1,000	999.9	55.00	540.5	yes
5,000	4,999.4	56.48	540.5	yes
10,000	9,998.3	58.81	540.5	yes

E. Detector Baseline Comparison & Quantization Degradation (H1)

Table VII compares our INT8 TCN–GRU against unsupervised, tabular and sequence baselines trained on the identical InSDN split; Transformer AE is a 2-layer encoder-decoder with 4 attention heads and masked reconstruction loss. Every model is scored at its own optimal threshold, since a threshold favouring one architecture need not suit another. *That threshold is selected by searching the test split*, so each $F1$ here is an optimistic bound rather than a deployable operating point; the identical search is applied to every model, so comparison between rows stays internally fair. The tree ensembles receive the window flattened to $T \times d = 100$ features — the same raw information the sequence models see — so the comparison does not handicap them.

Gradient-boosted trees beat the proposed detector, and this is our fifth negative result. LightGBM [22] reaches $F1 = 0.9886$ and XGBoost [23] 0.9880 against 0.9725 for TCN–GRU, and XGBoost does it at 0.387 ms per inference against 4.304 ms — higher accuracy at roughly a tenth of the latency. Random Forest also exceeds TCN–GRU (0.9843). These $F1$ values were bit-identical across every repeated run, so unlike our timings they are not measurement noise. On ten aggregate flow statistics, the temporal structure a TCN–GRU exists to exploit is evidently not where the discriminative signal lives. One advantage survives: serialized, the trees occupy 0.845 MB (XGBoost), 1.374 MB (LightGBM) and 20.562 MB (Random Forest) against 0.048 MB for the INT8 model — but the RQ2 budget was 1 MB and XGBoost fits inside it, so the footprint argument only becomes decisive an order of magnitude below the budget we specified. That is the same boundary condition we reached for quantization itself.

The Isolation Forest row uses `contamination="auto"` on time-averaged windows; on an 80%-attack corpus this heuristic is badly mismatched, so $F1 = 0.2000$ evidences that an untuned classical outlier detector fails at this class balance

rather than serving as a tuned baseline. Tree sizes above are serialized objects, not the parameter footprint of the table’s Size column.

INT8 quantization reduces the model footprint from 0.104 MB to 0.048 MB (53.8%) at *zero* measured accuracy cost ($\Delta F1 = 0.0000$ to four decimal places), supporting hypothesis **H1**. Counter-intuitively, dynamic INT8 on x86 CPU *increases* latency (4.304 ms vs 2.832 ms, a $1.52\times$ penalty) because activations are dequantized at every layer boundary without hardware INT8 acceleration: the technique buys memory, not speed, here. Since the RQ2 budget was 1 MB and the FP32 model already meets it at 0.104 MB, the honest reading of H1 is that quantization is *harmless* rather than necessary, and justified only for gateways an order of magnitude more constrained than ours.

TABLE VII

BASILINE & MODEL QUANTIZATION COMPARISON ON REAL INSDN. EACH MODEL IS SCORED AT ITS OWN $F1$ -OPTIMAL THRESHOLD SELECTED ON THE TEST SPLIT (SEE TEXT). ITALICS MARK MODELS THAT OUTPERFORM THE PROPOSED DETECTOR. SIZE IS THE PARAMETER FOOTPRINT, REPORTED ONLY FOR THE PYTORCH MODELS, FOR WHICH IT IS MEASURED ON A COMMON BASIS; “–” DENOTES A QUANTITY NOT MEASURED ON THAT BASIS.

Model Algorithm	F1-Score	Latency (ms)	Size (MB)
Isolation Forest (untuned)	0.2000	7.857	–
Random Forest	0.9843	46.301	–
XGBoost	<i>0.9880</i>	<i>0.387</i>	–
LightGBM	<i>0.9886</i>	0.711	–
LSTM Autoencoder (FP32)	0.8687	0.619	0.028
Transformer Autoencoder (FP32)	0.8680	1.175	0.068
TCN-GRU (FP32)	0.9725	2.832	0.104
TCN-GRU (INT8 Quantized)	0.9725	4.304	0.048

F. Component-Wise Ablation Study

Table VIII details the impact of removing components. Because each configuration scores on a different scale, we select the threshold on the training split and apply it unchanged to test; a shared fixed threshold would measure calibration mismatch instead of component contribution. InSDN has no identity annotations, so trust comes from a simulated overlay parameterised by $\rho = P(\text{low trust} \mid \text{attack})$, reported at $\rho = 0.7$.

The result contradicts our expectation. Identity fusion and the risk engine *do not* improve $F1$; removing them nudges it from 0.9602 up to 0.9680. What they buy is false-positive suppression: FPR falls from 25.19% under direct edge actuation to 12.05%, a 52% relative reduction. Since the controller autonomously isolates flows, trading 0.8 points of $F1$ for less than half the false alarms is operationally correct — but it is a trade, not a free gain. A sweep over $\rho \in [0.5, 0.9]$ (in the artifact) shows identity fusion improves $F1$ only once the overlay is strongly informative ($\rho \geq 0.8$), bounding what this mechanism can claim without real campus identity data.

G. Policy Control Benchmarks (RQ3)

Table IX benchmarks the cloud DQN controller against static and heuristic baselines on a single-step contention

TABLE VIII

SYSTEM COMPONENT ABLATION ANALYSIS ($\rho = 0.7$, THRESHOLD TUNED ON TRAIN). LATENCIES WERE MEASURED IN A SEPARATE RUN FROM TABLE IV; ONLY WITHIN-TABLE COMPARISON IS MEANINGFUL.

Architecture Configuration	F1-Score	FPR (%)	Latency (ms)
Full Proposed Pipeline	0.9602	12.05	6.644
— w/o Identity Context Fusion	0.9628	20.67	6.643
— w/o Cloud Risk Engine (Direct Edge Actuation)	0.9680	25.19	6.643
— w/o INT8 Quantization (FP32 Baseline)	0.9699	18.83	2.726
— w/o Redis Streams (Synchronous REST Call)	0.9602	12.05	7.654

environment whose reward $r(s, a)$ trades containment efficacy against collateral QoS damage and congestion relief. Because r is analytic, the optimal action a^* is obtained by exact enumeration over the four actions, giving a ground-truth oracle. The agent is trained by fitted Q-iteration over 4,000 environment interactions with ϵ -greedy exploration and evaluated on 5,000 held-out states.

The DQN selects the oracle-optimal action in 99.2% of states and attains essentially zero mean regret, against 38.2% for a static confidence ladder and 34.8% for a hand-tuned heuristic table. Two caveats temper this. First, the agent is trained on the same reward function the oracle maximises, so this shows function-approximation capacity within a known environment, not generalisation to a live campus. Second, counter to expectation, the DQN inflicts *more* raw QoS disruption on benign traffic (0.128) than the static ladder (0.045): reward-optimal behaviour is more aggressive than a conservative threshold rule, and the learned policy’s advantage lies in near-zero regret rather than gentleness.

TABLE IX

POLICY CONTROLLER BENCHMARK AGAINST A BRUTE-FORCE ORACLE ($n = 5,000$ HELD-OUT STATES). QoS DISRUPTION AND REGRET ARE LOWER-IS-BETTER.

Policy Mechanism	Latency (ms)	QoS Disr.	Regret	Action Acc. (%)
Static Rule Lookup	0.0004	0.045	0.080	38.2
Round-Robin Action Assignment	0.0003	0.187	0.205	25.5
Heuristic Risk Table	0.0004	0.052	0.072	34.8
Deep Q-Network (DQN Agent)	0.0942	0.128	0.000	99.2

H. How Much Did Leakage Inflate the Numbers?

Rather than assert our leakage controls were necessary, we measured what they are worth, rerunning the identical detector, seed, and schedule while changing only the split and scaler. Protocol A is the unsound one (i.i.d. split over overlapping windows, scaler fit on the pooled corpus); Protocol B is that of Section V-B.

The effect is real but small, and this is a negative result. Leakage inflates fixed-threshold $F1$ by only 0.0061 (0.9276 $\rightarrow$ 0.9337) and optimal-threshold $F1$ by 0.0021, far less than we expected given that 9 of every 10 timesteps cross the boundary. Two explanations fit and we cannot separate them: overlapping windows may be less informative than they look, since the one differing timestep is precisely the one carrying the transition; or this corpus may be easy enough that both protocols saturate

above 0.92. Our guidance is therefore narrower than we hoped — blocked-and-purged splitting is cheap and principled and we recommend it, but a reviewer should not assume a random split has by itself inflated a result by a large margin.

TABLE X
EFFECT OF THE EVALUATION PROTOCOL. IDENTICAL MODEL, SEED, AND SCHEDULE; ONLY THE SPLIT AND SCALER DIFFER.

Protocol	$F1 @ \tau = 0.65$	τ^*	$F1 @ \tau^*$
A: random split + pooled scaler (unsound)	0.9337	0.0945	0.9708
B: blocked + purged + train-only scaler	0.9276	0.0720	0.9687
Inflation (A – B)	+0.0061	—	+0.0021

I. Cross-Dataset Domain Shift Evaluation (RQ4)

Table XI evaluates zero-shot transfer from InSDN (14,000 train / 6,000 test windows) to a 20,000-window CIC-IDS2017 holdout. In-domain $F1$ is 0.9276 at $\tau = 0.65$; zero-shot $F1$ collapses to 0.4611. The failure mode is precision, not recall: the detector still recovers 92.8% of attacks but its false positive rate explodes to 92.4%, close to a degenerate always-attack classifier. Since detection is driven by the supervised head (Section VI-C), the *decision boundary* is what fails to transfer — CIC-IDS2017 benign traffic falls on the attack side of a boundary fitted to InSDN statistics — and re-tuning to $\tau^* = 0.0720$ does not help ($F1 = 0.4670$), so the degradation is representational. One factor we did not isolate: the corpora were feature-extracted under different tooling and flow-timeout conventions, so the 10 features may not be semantically identical.

TABLE XI
CROSS-DATASET EVALUATION: TRAIN INSDN $\rightarrow$ TEST CIC-IDS2017

Protocol	Threshold τ	Precision	Recall	F1-Score
InSDN $\rightarrow$ InSDN	0.65	0.9785	0.8818	0.9276
InSDN $\rightarrow$ InSDN	$\tau^* = 0.0720$	0.9469	0.9915	0.9687
InSDN $\rightarrow$ CIC-IDS2017	0.65	0.3068	0.9281	0.4611
InSDN $\rightarrow$ CIC-IDS2017	$\tau^* = 0.0720$	0.3060	0.9858	0.4670

VII. THREATS TO VALIDITY & LIMITATIONS

A. Threats to Validity

- **Internal Validity (measurement stability):** Timing on this host is noisy enough that single measurements are not results. Every timing figure is now a median over repeats — 5 for throughput and per-model latency, 1,000 trials summarised by percentiles for the control path — after we found that a single-run measurement had produced a throughput shortfall that repetition did not reproduce. Saturation throughput remains unstable between runs by more than a factor of two and we rest no claim on it. Accuracy metrics, by contrast, were bit-identical across all runs. Absolute latencies should be read as order-of-magnitude on this hardware, not as hardware-independent guarantees.

- **Reproducibility:** An earlier version of this artifact constructed models before seeding the global RNG, so detection metrics varied between runs of the same script. `set_global_seed` now runs at the head of every generator, before any module is built, making each script bit-deterministic. It is *not* re-invoked before each individual model, so a model built after several others inherits an RNG advanced by their construction — which is exactly why the same detector scores 0.9687 in Section VI-B and 0.9725 in Section VI-E. Determinism at a fixed seed is not robustness across seeds; accuracy is single-seed throughout.
- **External Validity:** Evaluation uses public InSDN and CIC benchmarks; broader IoT/IIoT corpora such as UNSW-NB15 [6], ToN-IoT [7], and Edge-IIoTset [8] offer complementary vectors for multi-campus validation.
- **Construct Validity:** Metrics assume clean flow labels; noisy headers could affect precision. We report no human-SOC comparison: we have no measured analyst baseline, and a stipulated one would establish only that milliseconds beat minutes, which needs no experiment.
- **Residual Leakage Risk:** Purging guarantees no shared timestep, but blocks are cut over the released record ordering, a proxy for session boundaries rather than ground-truth grouping, so a single long attack episode could straddle two blocks. Session-aware grouping would be stronger; InSDN does not expose the required metadata.

B. Limitations & Future Work

Primary limitations: (i) a single-controller emulated SDN testbed rather than production hardware; (ii) no evaluation of adversarial robustness against evasion, the obvious attack on any AI-native control loop; (iii) severe zero-shot cross-dataset degradation; (iv) rejection of H3 on its CPU term, the runtime costing 58.8% of a core against a 10% budget; (v) a detector outperformed on these features by off-the-shelf gradient boosting, leaving its case resting on footprint alone; (vi) an operating point that misses roughly one attack window in eight at $\tau = 0.65$, or trades that for a 12.1% full-pipeline false positive rate when tuned; and (vii) single-seed reporting of accuracy. Future work: domain adaptation, compiled ONNX/TensorRT edge runtimes, online threshold tuning, multi-seed reporting, and federated multi-campus learning.

VIII. ETHICS, PRIVACY & PUBLIC ARTIFACT

A. Privacy Posture

The architecture operates strictly on non-DPI flow headers; payloads are uninspected and IP/MAC identifiers are *pseudonymized* via SHA-256 before transport, supporting data minimization under GDPR and Vietnam’s Decree 13/2023. We avoid “anonymized”: an unkeyed SHA-256 over a 32-bit IPv4 or 48-bit MAC address is trivially invertible by enumeration. A keyed HMAC with a rotated per-deployment secret is the defensible construction; the unkeyed hash is a prototype placeholder, not a privacy guarantee.

B. Public Research Artifact

Code, preprocessors, checkpoints, Docker files, and scripts are available under MIT License at <https://github.com/thtsec/icai-fai2026>. Every table is regenerated by one entry point, `results/run_all_and_summarize.py`, which seeds all generators (seed 42) before model construction and emits the machine-readable summary the tables are transcribed from.

IX. CONCLUSION

This paper presented an AI-native event-driven edge-cloud architecture for autonomous campus network security. Combining INT8-quantized TCN-GRU edge detectors, Redis Streams transport, and cloud DQN SDN actuation, it achieves a software control path with a 2.681 ms median and a 5.764 ms p99, from telemetry ingestion to playbook emission. We stop short of calling this closed-loop mitigation: dataplane enforcement is unmeasured and the SOC comparison uses stipulated baselines.

The results we think transfer beyond this system are the negative ones. Gradient-boosted trees on the same features beat the proposed detector on accuracy and are an order of magnitude faster ($F1 = 0.9886$ for LightGBM against 0.9725; 0.387 ms for XGBoost against 4.304 ms), leaving the sequence model with only a footprint advantage that matters solely below a 1 MB budget. Reconstruction-error scoring, the default rule for sequence autoencoders, is unusable on an attack-heavy corpus ($F1 = 0.036$), so what we report is a supervised classifier with an auxiliary reconstruction objective and should be cited as such. Leakage from stride-1 overlap is real but, contrary to expectation, small ($\Delta F1 = 0.006$). H3 fails on its CPU term, sustaining 10,000 windows/s but at 58.8% of a core against a 10% budget. Zero-shot transfer to CIC-IDS2017 collapses to $F1 = 0.46$, driven by precision, locating the problem in a decision boundary that does not survive a change of capture domain. Together these bound where an AI-native campus defense is currently practical, and they point the architecture's value at integration and actuation rather than at the choice of detector.

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology at HUFLIT for academic and computational support.

REFERENCES

- [1] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurecut, "InSDN: A Novel SDN Intrusion Dataset," *IEEE Access*, vol. 8, pp. 165263–165284, 2020, doi: 10.1109/ACCESS.2020.3022633.
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018.
- [4] S. Kianpisheh and T. Taleb, "Collaborative Federated Learning for 6G With a Deep Reinforcement Learning-Based Controlling Mechanism: A DDoS Attack Detection Scenario," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, Aug. 2024, doi: 10.1109/TNSM.2024.3387987.
- [5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1–6, doi: 10.1109/MILCIS.2015.7348942.
- [7] A. Alsaedi *et al.*, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Cybersecurity Applications," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [8] M. A. Ferrag *et al.*, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022, doi: 10.1109/ACCESS.2022.3165809.
- [9] N. McKeown *et al.*, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [10] Ryu SDN Framework Community, "Ryu SDN Framework," 2020. [Online]. Available: <https://ryu-sdn.org/>
- [11] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.
- [12] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 2019, doi: 10.1109/COMST.2019.2904897.
- [13] T. A. Tang *et al.*, "Deep recurrent neural network for intrusion detection in SDN-based networks," in *Proc. 4th IEEE Conf. Netw. Softwarization (NetSoft)*, 2018, pp. 202–206, doi: 10.1109/NETSOFT.2018.8460090.
- [14] M. Roopak, G. Yun Tian, and J. Chambers, "Deep learning models for cyber security in IoT networks," in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, 2019, pp. 0452–0457, doi: 10.1109/CCWC.2019.8666588.
- [15] P. H. Nguyen *et al.*, "Towards Smarter Security Orchestration and Automatic Response for CPS and IoT," in *Proc. 14th IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, 2023, pp. 298–302, doi: 10.1109/CloudCom59040.2023.00055.
- [16] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010, pp. 1–6, doi: 10.1145/1868447.1868466.
- [17] M. B. Musthafa *et al.*, "Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices," *IEEE Access*, vol. 13, pp. 113544–113556, 2025, doi: 10.1109/ACCESS.2025.3584373.
- [18] S. Ullah *et al.*, "Quantized Intrusion Detection for Resource-Constrained IoT: A Comparative Evaluation of Efficiency and Adversarial Robustness," *Comput. Mater. Contin.*, vol. 88, no. 3, 2026, doi: 10.32604/cmc.2026.084409.
- [19] S. Srinivas *et al.*, "AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation," *J. Cybersec. Privacy*, vol. 5, no. 4, art. 95, 2025, doi: 10.3390/jcp5040095.
- [20] D. Arp *et al.*, "Dos and Don'ts of Machine Learning in Computer Security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.
- [21] M. A. Bouke and A. Abdullah, "An empirical assessment of ML models for 5G network intrusion detection: A data leakage-free approach," *e-Prime*, vol. 8, art. 100590, 2024, doi: 10.1016/j.prime.2024.100590.
- [22] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.
- [23] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [24] Y. Liu, M. Dong, K. Ota, J. Li, and J. Wu, "Deep Reinforcement Learning based Smart Mitigation of DDoS Flooding in Software-Defined Networks," in *Proc. IEEE CAMAD*, Barcelona, Spain, 2018, pp. 1–6, doi: 10.1109/CAMAD.2018.8514971.
- [25] N. M. Yungaicela-Naula, C. Vargas-Rosales, J. A. Pérez-Díaz, and M. Zareei, "Towards security automation in Software Defined Networks," *Comput. Commun.*, vol. 183, pp. 64–82, Feb. 2022, doi: 10.1016/j.comcom.2021.11.014.